

实验步骤

1.理论回顾与模型推导

推导 Logistic Regression 模型表达式：

$$h_{\theta}(x) = \frac{1}{1+e^{-\theta^T x}}, \text{ 其中 } \theta = [\theta_0, \theta_1, \dots, \theta_n]^T \text{ 是参数向量}$$

构造损失函数 (Log-Loss)：

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y_i \log(h_{\theta}(x_i)) + (1 - y_i) \log(1 - h_{\theta}(x_i))]$$

推导梯度并使用梯度下降进行参数更新

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}, \text{ 其中 } \frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x_i) - y_i) x_i^j$$

2.手动实现Logistic Regression

编写 Logistic Regression 类，包括：

- 初始化参数
- Sigmoid 函数、损失函数实现
- 模型训练-梯度下降实现
- 预测函数

可选：绘制损失函数下降过程、决策边界可视化

完成下面的类：

```
class LogisticRegression:
    """
    基于梯度下降法实现的对数几率回归模型 (Logistic Regression) 。

    参数:
        lr (float): 学习率，用于控制每次梯度更新的步长，默认值为 0.1。
        epochs (int): 最大迭代轮数，用于控制模型训练次数，默认值为 1000。
    """
    def __init__(self, lr=0.1, epochs=1000):
        self.lr = lr
        self.epochs = epochs

    def sigmoid(self, z):
        return

    def fit(self, X, y):
        """
        使用梯度下降训练对数几率回归模型。
        """
```

```

    参数：
        X (ndarray): 输入特征矩阵，形状为 (n_samples, n_features)。
        y (ndarray): 标签向量，形状为 (n_samples,)。
    """

    X = np.c_[np.ones((X.shape[0], 1)), X] # 添加偏置项
    self.theta = np.zeros(X.shape[1])      # 初始化参数

    self.losses = []

    for epoch in range(self.epochs):
        # 预测概率
        # 计算梯度
        # 更新参数

        # 计算当前轮的损失值（对数损失函数），为防止数值计算中的 log(0) 错误，使用log(x + 1e-8)

        self.losses.append(loss)

    def predict_proba(self, X):
        """
        预测给定输入样本属于正类的概率。

        参数：
            X (ndarray): 输入特征矩阵，形状为 (n_samples, n_features)。

        返回：
            ndarray: 每个样本属于正类的预测概率，值域为 [0, 1]。
        """
        # 添加偏置项

        return

    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(int)

```

训练模型并可视化：

```

# 训练模型

# 可视化损失
plt.plot(model.losses)
plt.title("Loss over Epochs")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()

```

可视化决策边界

```
def plot_decision_boundary(model, X, y):
    x_min, x_max = X[:, 0].min() - 0.05, X[:, 0].max() + 0.05
    y_min, y_max = X[:, 1].min() - 0.05, X[:, 1].max() + 0.05
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100),
                          np.linspace(y_min, y_max, 100))
    grid = np.c_[xx.ravel(), yy.ravel()]
    probs = model.predict(grid).reshape(xx.shape)

    plt.contourf(xx, yy, probs, cmap="RdYlBu", alpha=0.6)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap="bwr", edgecolors='k')
    plt.title("Decision Boundary")
    plt.xlabel("Density")
    plt.ylabel("Sugar Content")
    plt.show()

plot_decision_boundary(model, X, y)
```

3.使用Sklearn实现Logistic Regression

以鸢尾花数据集为例

Iris 鸢尾花数据集详见<https://gairuo.com/p/iris-dataset>

在线获取数据集链接<https://gairuo.com/file/data/dataset/iris.data>

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# 1. 加载 Iris 鸢尾花数据集 (共 3 类)
iris = load_iris()
X = iris.data[:, :2] # 取前两个特征, 方便二维可视化
y = iris.target      # 三类: 0 = Setosa, 1 = Versicolor, 2 = Virginica

# 2. 划分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 3. 创建并训练逻辑回归模型 (支持多分类)

# 4. 预测与评估
```

```

print("模型准确率: ", accuracy)
print("\n分类报告:\n", classification_report(y_test, y_pred,
target_names=iris.target_names))
print("混淆矩阵:\n", confusion_matrix(y_test, y_pred))

# 5. 可视化决策边界 (仅限前两个特征)
def plot_decision_boundary(model, X, y, title="Logistic Regression Decision Boundary"):
    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    h = 0.01 # 网格间隔
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    plt.figure(figsize=(8, 6))
    plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.Set3)
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Set3, edgecolors='k')
    plt.xlabel('Sepal length')
    plt.ylabel('Sepal width')
    plt.title(title)

    # 正确设置图例
    handles, _ = scatter.legend_elements()
    plt.legend(handles=handles, labels=list(iris.target_names))
    plt.show()

plot_decision_boundary(model, X, y)

```